



Universidad
Indoamérica

ユニット 5・オペレーティングシステムの基礎

パイプとプロセスの合成

Universidad Tecnológica Indoamérica 演習 TD4

隔離された2つのプロセスはどう通信するか？

バイトストリーム

パイプ（このユニット）、
FIFO（ユニット3）、
ソケット（ユニット8）

通知

シグナル（ユニット6）

共有メモリ

`mmap` と共有メモリ
（ユニット6）、スレッド
（ユニット7）

パイプは最も単純で最もよく使われる仕組み：シェルの `|` はすべてパイプ。

パイプ： pipe(fds)

- カーネル内にバッファと2つのディスクリプタを作る：読み込み用 `fds[0]` と書き込み用 `fds[1]`
- 一方向で、メッセージの区切りも名前もないバイトストリーム
- 共有できるのはディスクリプタを受け継いだプロセスだけ：`fork` の前に作る
- 各プロセスは**使わない端を閉じる**

C tds/td4/mypipe.c

```
exit_if(pipe(fds) == -1, "pipe");
pid_t pid = fork();
exit_if(pid == -1, "fork");

if (pid == 0) {
    exit_if(close(fds[1]) == -1, "close");
    char buffer[READ_MAX];
    ssize_t n = read(fds[0], buffer, READ_MAX);
    exit_if(n == -1, "read");
    exit_if(write_all(STDOUT_FILENO, buffer, (size_t)n)
    fprintf(stderr, "I am the child, I read %zd bytes\n", n);
    exit_if(close(fds[0]) == -1, "close");
    return EXIT_SUCCESS;
```

パイプの規則

空のパイプから読む	ブロックする...書き手が1つも残っていなければ 0 を返す
満杯のパイプに書く	誰かが読むまでブロックする
読み手がまったくいないのに書く	SIGPIPE シグナル（無視していれば EPIPE ）
PIPE_BUF （4096 B）以下を書く	アトミック：他の書き込みと混ざらない

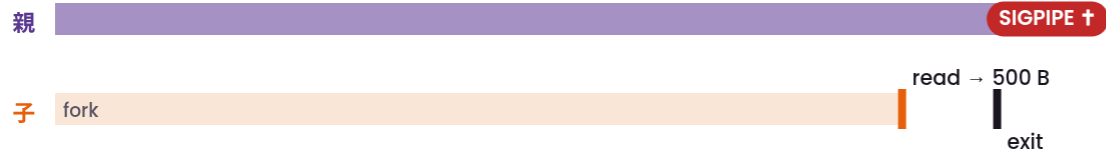
授業用マシンでの既定の容量：

実際の出力

```
$ python3 -c 'import os, fcntl; r, w = os.pipe();  
print(fcntl.fcntl(w, fcntl.F_GETPIPE_SZ), "bytes")'  
65536 bytes
```

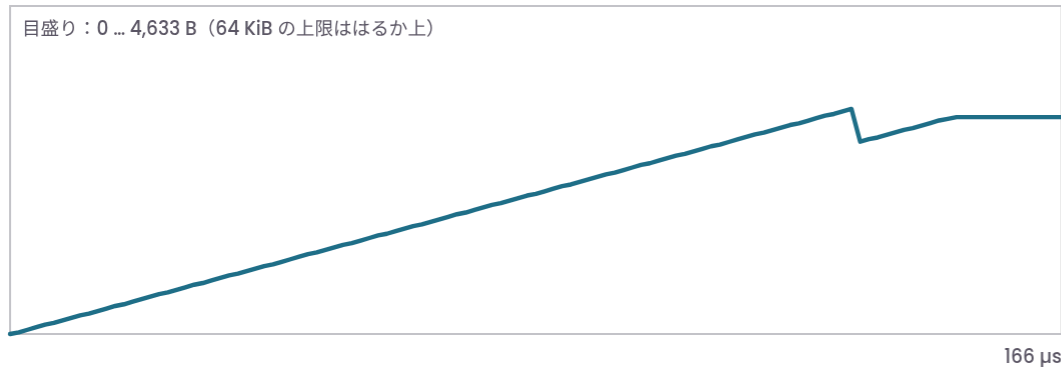
10、300、10,000 メッセージ：何が起きる？

タイムライン (モデル、 μs)

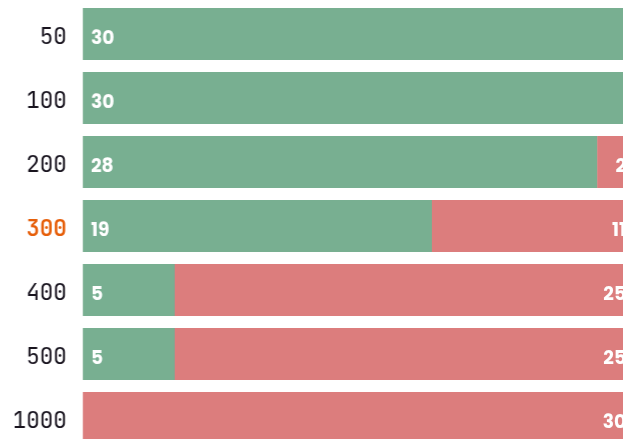


パイプ内のバイト数

目盛り：0 ... 4,633 B (64 KiB の上限ははるか上)



実測：サイズごとに 30 回実行



緑：ステータス 0・赤：141 (SIGPIPE)
メッセージ

親で `fds[0]` を閉じるか閉じないか

実際の出力

```
$ ./mypipe 10000 > /dev/null; echo "exit status of mypipe: $?"  
I am the child, I read 500 bytes and I leave  
exit status of mypipe: 141
```

実際の出力

```
$ timeout 3 ./mypipe_bug 10000 > /dev/null; echo "exit status: $? (124 = killed by timeout: it was blocked)"  
I am the child, I read 500 bytes and I leave  
exit status: 124 (124 = killed by timeout: it was blocked)
```

端を閉じていれば、子が去ったとき親は SIGPIPE を受ける (141 = 128 + 13)。閉じないと親自身が「読み手」になり、64 KiB を満たして永遠にブロックする (124 = `timeout` による強制終了)。`&` 付きで実行すると、シェルが終了ステータスを報告する。

他人のプログラムをつなぐ： `dup2` + `exec`

C tds/td4/upper.c

```
/* parent */
exit_if(close(fds[0]) == -1, "close");
exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2");
exit_if(close(fds[1]) == -1, "close");          /* 1 already closed */
execlp("ps", "ps", (char *)NULL);              /* "ps" to be executed */
exit_if(1, "execlp ps");
```

実際の出力

```
$ ./upper | head -4
  PID TTY          TIME CMD
 6706 ?           00:00:25 UVICORN
 9771 ?           00:00:03 SYSTEMD
 9801 ?           00:00:00 (SD-PAM)
```

`ps` はパイプのことを何も知らない：ディスクリプタ 1 に書くだけで、それを `dup2` がパイプにつないだ。子はファイル終端まで読み、大文字に変換する。

シェルが `ls -l | tr -d '[:blank:]'` で行うこと

1. `pipe(fds)`
2. `fork` → 子 1: `dup2(fds[1], 1)`、両方を閉じ、`exec("ls")`
3. `fork` → 子 2: `dup2(fds[0], 0)`、両方を閉じ、`exec("tr")`
4. シェルは自分の両端を閉じ、2 つの子を待つ

ディスクリプタ表を一步步

pipeline

pipeline

0	端末
1	端末
2	端末
3	
4	
5	
6	

子1

ls -l demo

0	端末
1	パイプ0・書き
2	端末
3	
4	
5	
6	

子2

grep txt

0	パイプ0・読み
1	パイプ1・書き
2	端末
3	
4	
5	
6	

子3

wc -l

0	パイプ1・読み
1	端末
2	端末
3	
4	
5	
6	

パイプ0・書き手1・読み手1

パイプ1・書き手1・読み手1

行 60: close(4)

N 個のコマンドとパイプラインのステータス

C tds/td4/pipeline_args.c

```
if (pid == 0) {
    if (in != STDIN_FILENO) {
        exit_if(dup2(in, STDIN_FILENO) == -1, "dup2");
        exit_if(close(in) == -1, "close");
    }
    if (fds[1] != -1) {
        exit_if(dup2(fds[1], STDOUT_FILENO) == -1,
            exit_if(close(fds[0]) == -1 || close(fds[1])
        }
        execvp(commands[i][0], commands[i]);
        perror(commands[i][0]);
        _exit(127);
    }
    if (in != STDIN_FILENO)
        exit_if(close(in) == -1, "close");
    if (fds[1] != -1) {
        exit_if(close(fds[1]) == -1, "close");
        in = fds[0];
    }
}
```

実際の出力

```
$ ./pipeline_args ls -l demo -- grep txt -- wc -l
2
```

実際の出力

```
$ ./pipeline_args ls -- sh -c 'exit 3'; echo "status: $?"
status: 3
```

実際の出力

```
$ ./pipeline_args ls -- sh -c 'kill -TERM $$'
pipeline: last command killed by SIGTERM
[exit status 1]
```

シェルと同じく、**最後の**コマンドのステータスを返す
(bash では `set -o pipefail` でこの規則が
変わる)。

pipe の先へ

pipe2

`O_CLOEXEC` は `exec` で起動したプログラムにパイプが漏れるのを防ぐ。

`O_NONBLOCK` はブロックしないため。

splice

データをプロセスにコピーせずに、パイプとファイルやソケットの間で移す。

容量

`fcntl(F_SETPIPE_SZ)` で `/proc/sys/fs/pipe-max-size` まで大きくできる。

解答付き TD4 + AI チャレンジ

- 10、300、10,000 メッセージの `mypipe` : バグありとなし
- `upper` : 大文字の `ps`
- `pipeline_args ls -l -- grep txt -- wc -l`

Bash

```
make -C tds && tds/tests/run.sh td4
```



Universidad
Indoamérica

まとめ

3つの問い

1. パイプへの `read` が 0 を返すのはいつ？
2. `mypipe 10000` が 141 で終わり、`mypipe_bug` が終わらないのはなぜ？
3. `a | b | c` で、シェルは `pipe`、`fork`、`close` を何回呼ぶ？